Proyecto Administración de Solicitudes

Documento de Análisis y Diseño

Estado actual: Primer Sprint – Aplicación Web (monolito)

Administración y Programación de Sitios Web

Colegio Universitario de Cartago

Prof. Ing. Gabriel González Solano

Grupo: Fuerzas Especiales Ginyû

Sebastián Jiménez Arrieta
Aldo Burgos Argüello
Cristian Esteban Monge Raabe
Dorian Blanco Calderón

Cartago, Costa Rica
30 de septiembre de 2026

Tabla de contenido

1. Introducción 2
2. Arquitectura general del sistema 3
3. Diagrama de casos de uso 4
4. Diagrama de clases 5
5. Diagrama de base de datos 6
6. Conclusiones y recomendaciones 7
6.1 Conclusiones 7
6.2 Recomendaciones 7
7. Bibliografía 8

1. Introducción

La empresa Pública S.A. gestiona actualmente sus solicitudes mediante archivos Excel, lo que dificulta el seguimiento, el control y la colaboración entre los distintos departamentos y genera problemas como falta de trazabilidad, errores en el control de estados, pérdida de información y demoras en la atención. Para atender esa necesidad se propone el Proyecto Administración de Solicitudes, desarrollado durante el curso de Administración y Programación de Sitios Web del Colegio Universitario de Cartago.

El proyecto consiste en desarrollar un sistema web que automatice este proceso, mediante una plataforma centralizada y moderna para la gestión integral de solicitudes. El sistema permite registrar solicitudes, asignar tareas, gestionar sus estados, generar reportes y realizar seguimientos de manera automatizada, e incorpora notificaciones por correo electrónico, bitácoras de auditoría y un modelo de seguridad basado en usuarios. La solución se construye como un monolito: una única aplicación web que reúne la interfaz, la lógica de negocio y el acceso a datos, conectada a una sola base de datos.

El alcance de este primer sprint comprende 14 historias de usuario, organizadas en los siguientes módulos:

- **Usuarios:** pantalla de login, pantalla de bienvenida, plantilla global del sitio y administración de usuarios.
- **Solicitudes:** administración de solicitudes, desgloses, representantes y estados, el proceso automático de vencimiento, la notificación por correo de los cambios de estado y el reporte de montos a facturar por mes.
- **Tareas:** registro de tareas asociadas a las solicitudes y reportes de tareas por usuario y por solicitud.

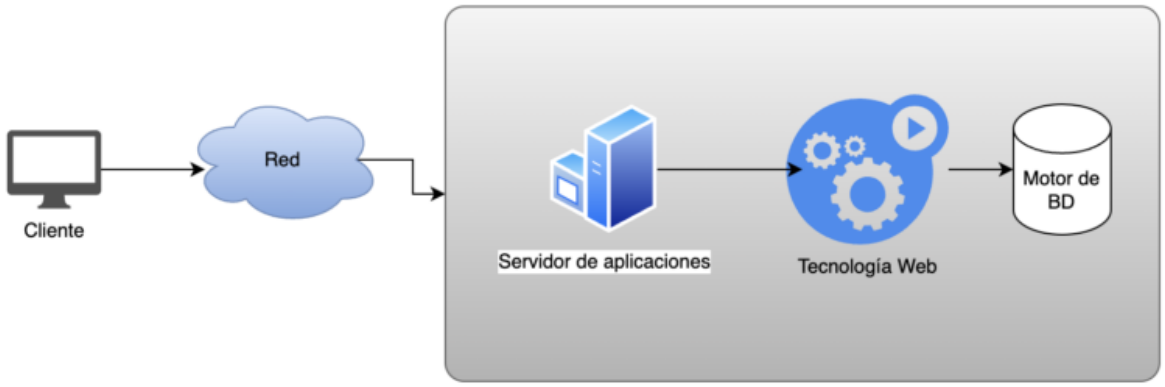
Para su desarrollo, el equipo trabaja bajo el modelo Scrum y utiliza ASP .NET 8 para la aplicación web, MySQL como base de datos, Dapper para el acceso a datos y Bootstrap para la interfaz, junto con un repositorio Git con una estructura de ramas Main, Develop y Feature, cuyos cambios se integran mediante Pull Requests.

El presente documento reúne el análisis y diseño de la solución: la arquitectura general, el diagrama de casos de uso, el diagrama de clases y el diagrama de la base de datos, así como las conclusiones y recomendaciones del equipo. La evidencia de las pruebas técnicas se presenta en un documento independiente dentro del repositorio oficial del proyecto.

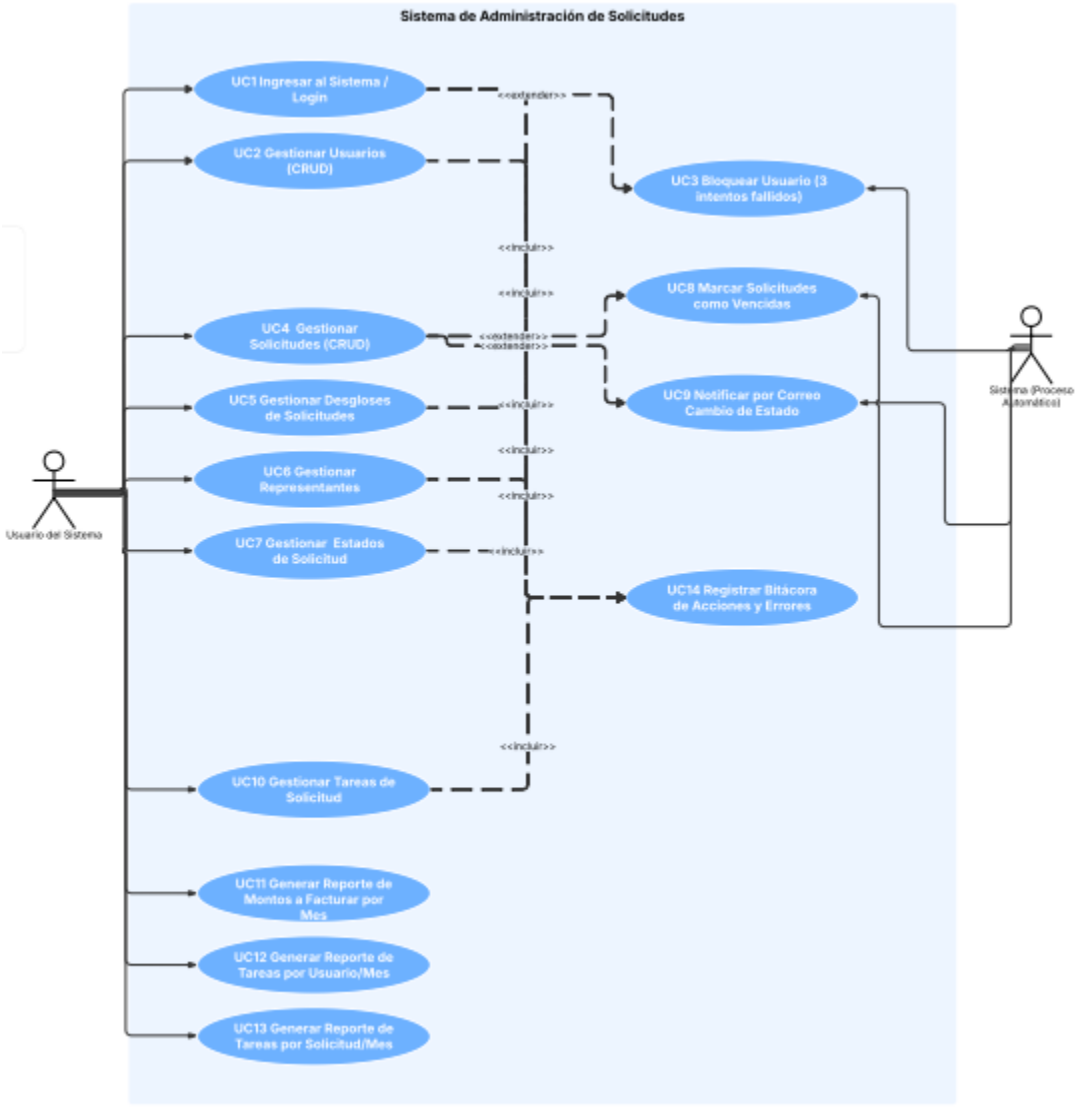
2. Arquitectura general del sistema

El sistema se construye como una aplicación monolítica: toda la funcionalidad (usuarios, solicitudes y tareas) se despliega como una sola unidad y comparte el mismo proceso y la misma base de datos, por lo que no existen microservicios ni servicios REST independientes. A nivel general, la arquitectura se compone de los siguientes elementos:

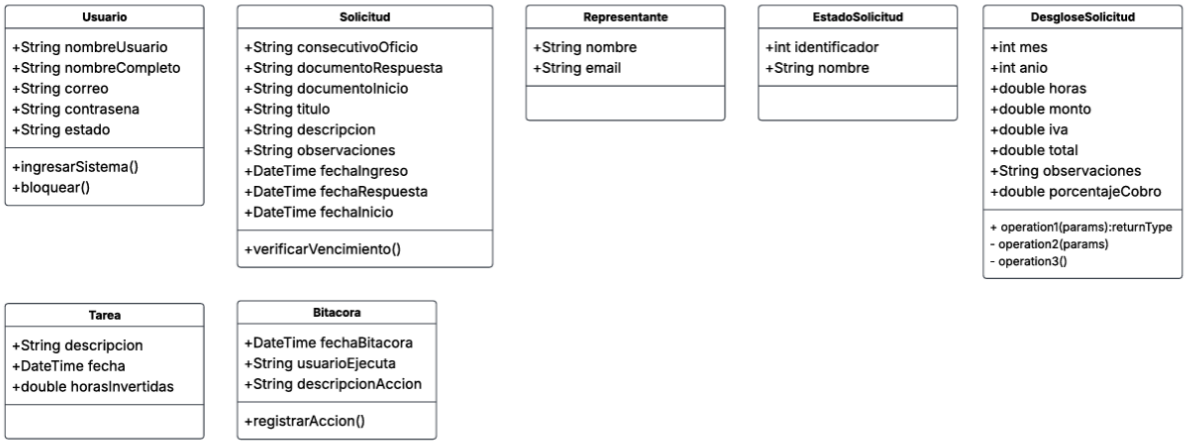
- **Cliente:** navegador web desde el cual los usuarios acceden al sistema a través de la red.
- **Servidor de aplicaciones:** aloja la aplicación web monolítica, que atiende las pantallas, los procesos automáticos y los reportes.
- **Tecnología web:** ASP .NET 8 con Bootstrap para la interfaz y Dapper para el acceso a datos.
- **Motor de base de datos:** MySQL, con una única base de datos para todo el sistema.



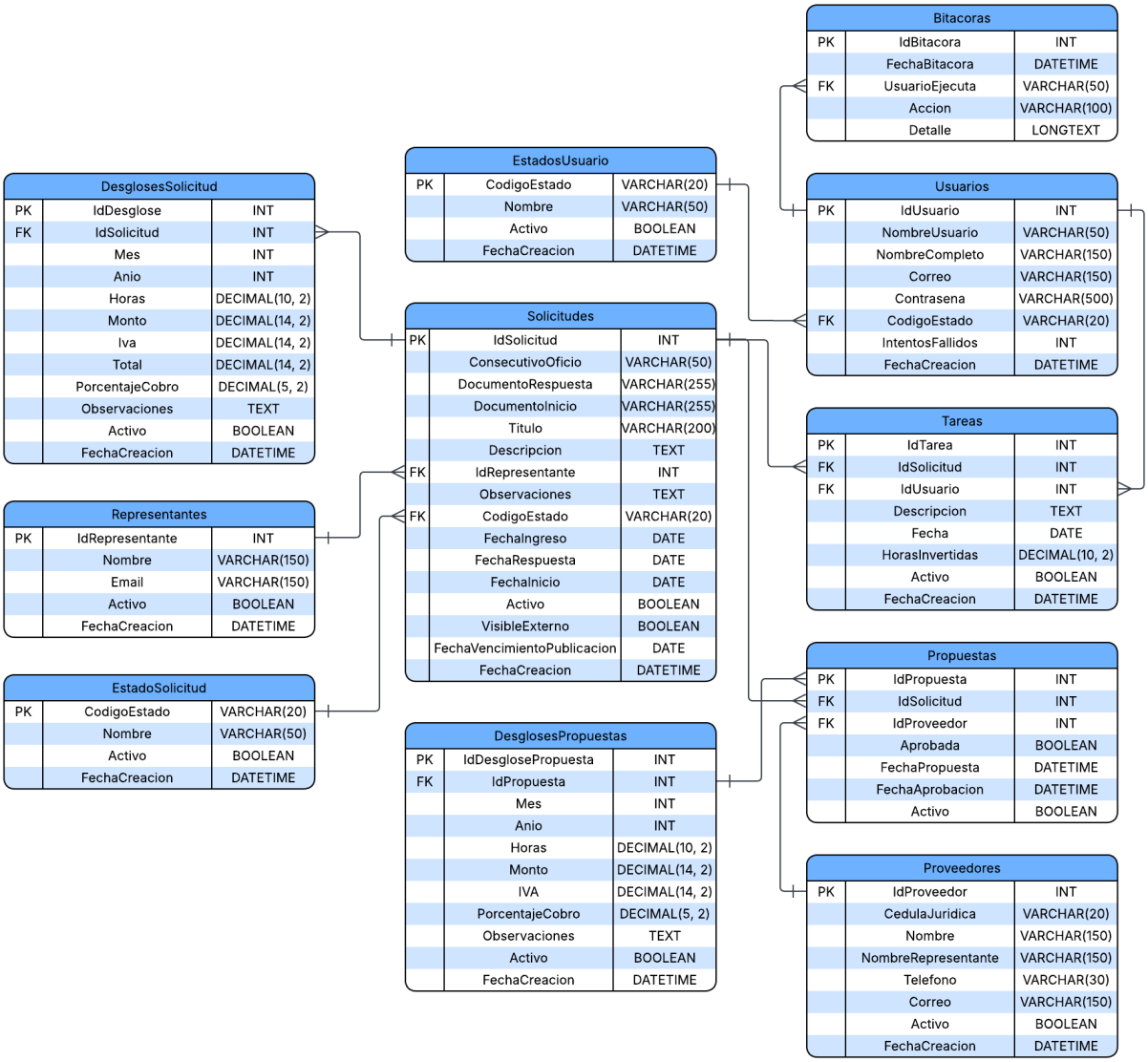
3. Diagrama de casos de uso



4. Diagrama de clases



5. Diagrama de base de datos



6. Conclusiones y recomendaciones

6.1 Conclusiones

1. El sistema web centraliza la gestión de solicitudes de Pública S.A., que hoy se hace en Excel. Esto resuelve la falta de trazabilidad, los errores en el control de estados y la pérdida de información.
2. Se cubrieron las 14 historias de usuario del primer sprint, distribuidas en los módulos de Usuarios, Solicitudes y Tareas.
3. La arquitectura monolítica (una sola aplicación y una sola base de datos) fue adecuada para este alcance. Simplificó el desarrollo, las pruebas y el despliegue.
4. ASP .NET 8, Dapper, Bootstrap y MySQL permitieron construir una solución funcional con una estructura clara, separada en capas.
5. Las notificaciones por correo, las bitácoras de auditoría y el proceso automático de vencimiento dan seguimiento y control sobre cada solicitud.
6. Scrum, el repositorio Git con ramas Main, Develop y Feature, y los Pull Requests ayudaron a organizar el trabajo del equipo y a mantener el código ordenado.
7. Los diagramas de casos de uso, clases y base de datos dejan documentada la solución y sirven de base para los siguientes sprints.

6.2 Recomendaciones

1. Mantener las capas bien separadas (entidades, servicios y acceso a datos), para que el sistema siga siendo fácil de mantener y de ampliar.
2. Revisar los diagramas cuando cambie el código o la base de datos, para que la documentación no quede desactualizada.
3. Reforzar las validaciones y restricciones en la base de datos, además de las que ya existen en la aplicación, para proteger la integridad de los datos.
4. Mantener las bitácoras de auditoría en todas las funcionalidades nuevas, y no solo en las del primer sprint.
5. Ampliar las pruebas técnicas conforme se agreguen funcionalidades, y dejar la evidencia en el repositorio.
6. Seguir usando Pull Requests con revisión del equipo antes de integrar a Develop y a Main.
7. Si el sistema crece mucho en el futuro, evaluar separar módulos en servicios independientes. Para el alcance actual, el monolito sigue siendo suficiente.
8. Hacer el despliegue en IIS con una configuración documentada, para poder repetirlo sin errores.

7. Bibliografía

No aplica en nuestro caso.